BioHackathon series:
Eurobioc 2026
Turku, Finland 2026
Project 2

Submitted: 02 Jun 2026

Published by BioHackrXiv.org

BiocExecute: Make package functions or workflows executable in the command line

Guillaume Deflandre [1], **Leopold Guyot** [1], **Rasmus Hindström** [2], **and Claire Rioualen** [3]

1 Computational Biology and Bioinformatics, de Duve Institute, UCLouvain **2** Department of Computing, University of Turku, Turku, Finland **3** IFB-core, French Institute of Bioinformatics (IFB), CNRS, INSERM, INRAE, CEA, 94800 Villejuif, France

Introduction

Commandline executables for bioconductor functions and scripts

This project was originally envisioned as a package or function for converting package maintainer or user supplied scripts into commandline executables. The R and Bioconductor paradigms of interactivity through a responsive and informative REPL have been a formula for success for academic users for a long time. But as computational biology becomes increasingly interdisciplinary and increasingly relies on high performance compute, high throughput compute and experimentation, and cloud compute, formalizing portable and flexible implementations of R and Bioconductor tooling is an imperative to ensure that that work, and Bioconductor itself maintain relevancy.

There already exist at least two tools that do 'app' style script conversion; R2G2 - Galaxy specifc integration and Rapp - commandline executables. So developing a package for this project may not be necessary. Long term, the goal of this project is to lay the groundwork for programmatic generation of tooling within Bioconductor packages that can be slotted into modern workflow management systems, or other interactive platforms like Galaxy.

Key Considerations:

- Where is the right place for 'app-ification' of a script to take place? Should it happen upon package installation?
- Overhead checking of things like package versions or argument types are mostly cheap, how much of those checks should we enforce?
- What do graceful failures for these scripts look like?

Motivation

Bioconductor has a collection of more than 2,400 open source software packages downloaded millions of times per year. They are thoroughly maintained and documented, and their quality is ensured through the use of BiocCheck.

This package aims to further improve Bioconductor software FAIRness, by making them usable in the command line.

- Findability: xxx;
- Accessibility: allows more users to use Bioconductor packages, in a wider variety of setups;
- Interoperability: packages can be combined as modules with other command line tools;

- Reusability: modular components can be reused in future workflows with any workflow manager.

Package users and developers can both benefit from this setup: users can use Bioconductor tools out side of R scripts and integrate them in their own workflows, and package developers can benefit from a wider range of users. Overall, Bioconductor software can gain visibility among a larger community of bioinformaticiens.

Light weight, relies on existing packages

Results

BiocExecute

`BiocExecute` is a package to make Bioconductor/R package functions executable in the Command Line Interface (CLI) (Figure 1.)

Figure 1: Hex sticker for the BiocExecute package.

The package comes with a few functions that need to be called upon building a package or upon using the package in the CLI.

How to use executables

Here's a quick example of how you would call the function `name(x, y)` from the package `pkgExample`:

First, the user needs to make sure the package functions are executable:

```
BiocExecute::installExecs("pkgExample")
```

And now call those functions from within the terminal:

```
pkgExample --help
pkgExample name -x Bilbo -y Baggins
```

How to create executables

`BiocExecute` uses `Rapp` to make your package functions executable. In the coming sections we will show the different ways `Rapp` includes arguments to be called in the CLI. Note that `Rapp` by itself works with scripts in which the first line is defined as `#!/usr/bin/env Rapp`. You should ***NOT*** include this line in your scripts ! This is handled internally as it is bundled in the `BiocExecute` package.

The executables can have many ranges, from a simple function call to an entire complex workflow. Note that bigger workflows mean more parameters to call in the CLI.

As a maintainer, you may choose which functions/workflows you decide to include in your package. As a package user, we suggest to create pull requests on the package GitHub repository for workflows that you believe should be easily accessible. Try and avoid creating strongly personal workflows. Hold priority for workflows that are repeatedly used across different user cases.

Create a script

An *R* package that has executables should include them in its `exec/scripts/` directory. `BiocExecute` has all the necessary functions to create these folders, scripts and more.

For instance, suppose the name of my package is `mypkg`. In its root directory, I don't have an `exec` nor a `scripts` folder:

```
mypkg
|   README.md
|   DESCRIPTION
|   NEWS.md
|   NAMESPACE
|
+---R
|   |   functionA.R
|   |   functionB.R
|
+---tests
|   |   testA.R
|   |   testB.R
|
+---vignettes
    |   myVignette.Rmd
```

To create the necessary files, I use:

```
{r createSkeleton, eval = FALSE} ## From the root directory execSkeleton(
)
```

Now, my directory tree looks like this:

```
mypkg
|   README.md
|   DESCRIPTION
|   NEWS.md
|   NAMESPACE
|
+---R
|   |   functionA.R
|   |   functionB.R
|
```

```
+---exec
|   |   mypkg.R
|   |
|   +---scripts
|       |   base_template.R
|
+---tests
|   |   testA.R
|   |   testB.R
|
+---vignettes
    |   myVignette.Rmd
```

For now, there is a simple template in the `scripts` folder. The same template can be created using `execTemplate()`.

The maintainer of a package should then create the scripts with functions or workflows that he/she wishes to make executable on the CLI.

Once the `exec/scripts/` are created, you should (in order from the root directory):

1. Call `execCompile()` to build the actual executable script. This file will be called by its package name and it will be written in the `exec` folder. Do not edit this file by hand, it is likely to be overwritten.
2. Call `devtools::install()` to re-install the package with the executables.
3. Call `execInstall("mypkg")` or a vector of packages to make the executables available in the CLI. To remove those, call `execUninstall("mypkg")`. To make the executables available system-wise (with sudo access), use the `destdir` parameter.

Your package functions/tools are now available in the CLI !

Script files

The files in the `exec/scripts/` directory are at the core of the available executables. One script correcsponds to one command, which can be a simple function or even a whole workflow. These scripts are combined and compiled into the main executable file in `exec/` (see section above).

The script files need to follow the `Rapp` architecture. A template file is built by default to get you started. In practice, these *R* scripts are really a repetition of the important functions within your package, except that their parameters and documentation are re-written in a way for `Rapp` to parse them.

Rapp fields and function parameters

Rapp fields and function parameters

`Rapp` parses *R* scripts by looking for specific expression patterns at the top level. Each pattern maps to a different CLI surface. The table below summarises the most common ones:

R expression	CLI surface
`foo <- ""`	Option: `app --foo value`
`foo <- NULL`	Positional argument: `app foo-value`
`foo <- TRUE`	Boolean switch: `app --foo` / `app --no-foo`
`foo <- c()`	Repeatable option (raw strings): `app --foo a --foo b`
`foo <- list()`	Repeatable option (parsed values): `app --foo 1 --foo 2`
`switch("", cmd1 = {}, cmd2 = {})`	Subcommands: `app cmd1 --help`

Annotations are written as YAML hash-pipe comments (`#|`) directly above the assignment they document. The most commonly used fields are:

Field	Description
`description`	Short description shown in `--help` output
`title`	Title for subcommands
`short`	Single-letter alias (e.g. `short: n` enables `-n`)
`required`	Set to `false` to make a positional argument optional
`val_type`	Expected type: `string`, `integer`, `float`, `bool`, or `any`

A minimal example script illustrates how these come together:

```
#| description: Count word occurrences in a file.

#| description: Path to the input file.
inputFile <- NULL

#| description: Word to count.
#| short: w
word <- ""

#| description: Print each match.
verbose <- FALSE
```

Called from the terminal:

```
count-words myfile.txt --word hello --verbose
count-words --help
```

For a full description of all available fields and advanced patterns such as nested subcommands, refer to the Rapp GitHub page.

Data availability

All scripts and materials developed during the hackathon are available in the BiocExecute GitHub repository.

Conclusion?

Acknowledgements

This work received state aid managed by the National Research Agency under France 2030 for the French Institute of Bioinformatics (IFB), founded by the Investments for the Future Program, number ANR-11-INBS-0013, as well as for structural research equipment / EQUIPEX+ with reference ANR-21-ESRE-0048.

References

Author contributions

Guillaume Deflandre: Author, creator Leopold Guyot: Author Rasmus Hindström: Author Claire Rioualen: Author